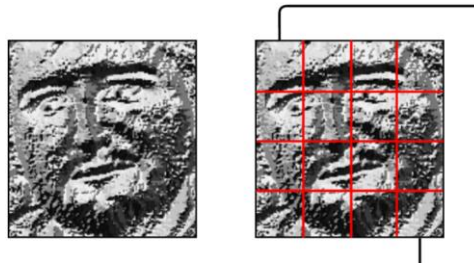
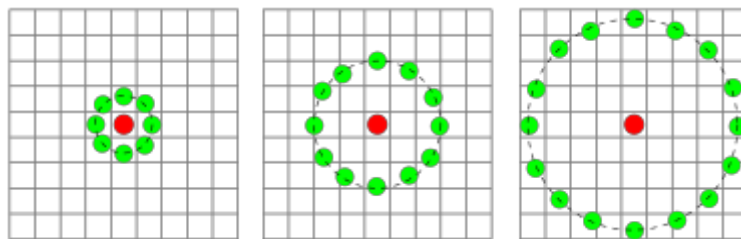


Local Binary Pattern

The LBP feature vector, in its simplest form, is created in the following manner:



- Divide the examined window into cells (e.g. 16x16 pixels for each cell).
- For each pixel in a cell, compare the pixel to each of its 8 neighbors (on its left-top, left-middle, left-bottom, right-top, etc.). (BASED ON THE Radius (distance from center) values)
- Follow the pixels along a circle, i.e. clockwise or counter-clockwise.
- In LBP, we define the number of neighbors (P) and the radius (R). For example, (P=8, R=1) means 8 pixels at a distance of 1 pixel are used. Larger radius captures coarser textures (e.g., (P=16, R=2)).
- Where the center pixel's value is greater than the neighbor's value, write "0". Otherwise, write "1". This gives an 8-digit binary number (which is usually converted to decimal for convenience).
- Compute the histogram, over the cell, of the frequency of each "number" occurring (i.e., each combination of which pixels are smaller and which are greater than the center). This histogram can be seen as a 256-dimensional feature vector.
- Optionally normalize the histogram. # Normalization ensures that lighting variations (bright or dark regions) don't affect the texture description
- Concatenate (normalized) histograms of all cells. This gives a feature vector for the entire window.



(the pixel values lying on the boundary, new Gray values are calculated using interpolation)

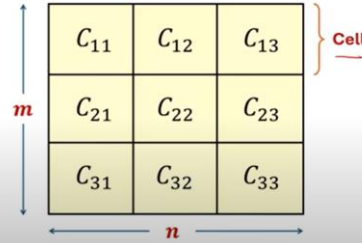
$$LBP_{p,r} = \sum_{p=0}^{p-1} s(g_i - g_c) 2^i, S(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Let image size is $m \times n$ and cell size is $a \times b$, the number of cells will be,

$$\text{numCells} = \left(\text{floor}\left(\frac{m}{a}\right) \times \text{floor}\left(\frac{n}{b}\right) \right)$$

It $m = n = 200$ and $a = b = 64$ then
Number of cells,

$$\left(\frac{200}{64} \times \frac{200}{64} \right) = (3.125 \times 3.125) = (3 \times 3) = 09$$



Calculation of LBP

1. **Whole image as one cell** (global LBP)
2. **Dividing the image into smaller cells/blocks** (cell-wise or spatial LBP)

Whole Image LBP

- The **entire image** is treated as a **single cell**.
- This gives a **global texture description** of the image.
- Limitation: Loses **spatial information** (where textures occur).

Cell-wise LBP

- The image is **divided into small grids** (e.g., 8×8 or 16×16 cells).
- Compute an **LBP histogram** for each cell.
- Concatenate all histograms $\rightarrow$ final feature vector.
- Captures **local texture variations** and **spatial layout** — much better for face or object classification.

50	80	70
60	65	55
40	45	50

Here,

the **center pixel** $gc=65$. We'll compute the LBP value of this **one pixel**, treating the entire patch as our image (whole-image LBP).

We have $P=8$ neighbors, indexed clockwise starting from the **top-left**.

i (neighbor)	gi	$(gi - gc)$	$s(gi - gc)$	Weight (2^i)	Contribution
0 (top-left)	50	$50 - 65 = -15$	0	128	0
1 (top)	80	$80 - 65 = 15$	1	64	64
2 (top-right)	70	$70 - 65 = 5$	1	32	32
3 (right)	55	$55 - 65 = -10$	0	16	0
4 (bottom-right)	50	$50 - 65 = -15$	0	8	0
5 (bottom)	45	$45 - 65 = -20$	0	4	0
6 (bottom-left)	40	$40 - 65 = -25$	0	2	0
7 (left)	60	$60 - 65 = -5$	0	1	0

Step 3: Binary Pattern and Decimal Value

Binary pattern ($g_0 \rightarrow g_7$):

0 1 1 0 0 0 0

Now convert to decimal:

LBP code = 96

Step 4: Whole Image LBP Histogram

If this 3×3 patch is your **entire image**, we only have one center pixel $\rightarrow$ one LBP code = **6**.

So the **LBP histogram** (for 8-bit uniform pattern) would have 59 bins (0–58 uniform + 1 non-uniform).

In this case, only the **bin for 6** has count **1**.

LBP value	Count
96	1
All others	0

That's the **whole image LBP feature**: a compact global texture description.

Numerical: Compute LBP for whole Image

40	45	50	60
30	35	55	65
25	30	60	70
20	25	50	55

Step 2: Compute LBP for Each Valid Center

We can only compute LBP for pixels that have a **3×3 Neighborhood** — i.e., not the border pixels.

So, valid centers are at positions:

(1,1), (1,2), (2,1), (2,2) $\rightarrow$ total 4 centers.

i (neighbor)	g_i	$(g_i - g_c)$	$s(g_i - g_c)$	Weight (2^i)	Contribution
0	40	+5	1	128	128
1	45	+10	1	64	64
2	50	+15	1	32	32
3	55	+20	1	16	16
4	60	+25	1	8	8
5	30	-5	0	4	0
6	25	-10	0	2	0
7	30	-5	0	1	0

Center (1,1): $g_c = 35$

Neighbors: [40,45,50,55,60,30,25,30]

{1,1,1,1,1,0,0,0}= LBP = 248

Center (1,2): gc = 55

Neighbors: [45,50,60,65,70,60,30,35]

| gi - gc | -10 | -5 | +5 | +10 | +15 | +5 | -25 | -20 |
| s | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |

Binary pattern = **0 0 1 1 1 1 0 0**

→ LBP = 128+64+32+16= 240

Center (2,1): gc = 30

Neighbors: [30,35,55,60,70,50,20,25]

Similarly, Calculate LBP for this center

Binary pattern = **1 1 1 1 1 1 0 0**

→ LBP =252

Center (2,2): gc = 60

Neighbors: [35,55,65,70,55,50,25,30]

| gi - gc | -25 | -5 | +5 | +10 | -5 | -10 | -35 | -30 |
| s | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |

Binary = **0 0 1 1 0 0 0 0**

→ LBP = 48

Center (r,c)	LBP Code
(1,1)	248
(1,2)	240
(2,1)	60
(2,2)	48

Step 4: Build the Histogram

Now we build a **histogram** counting how many times each LBP value appears.

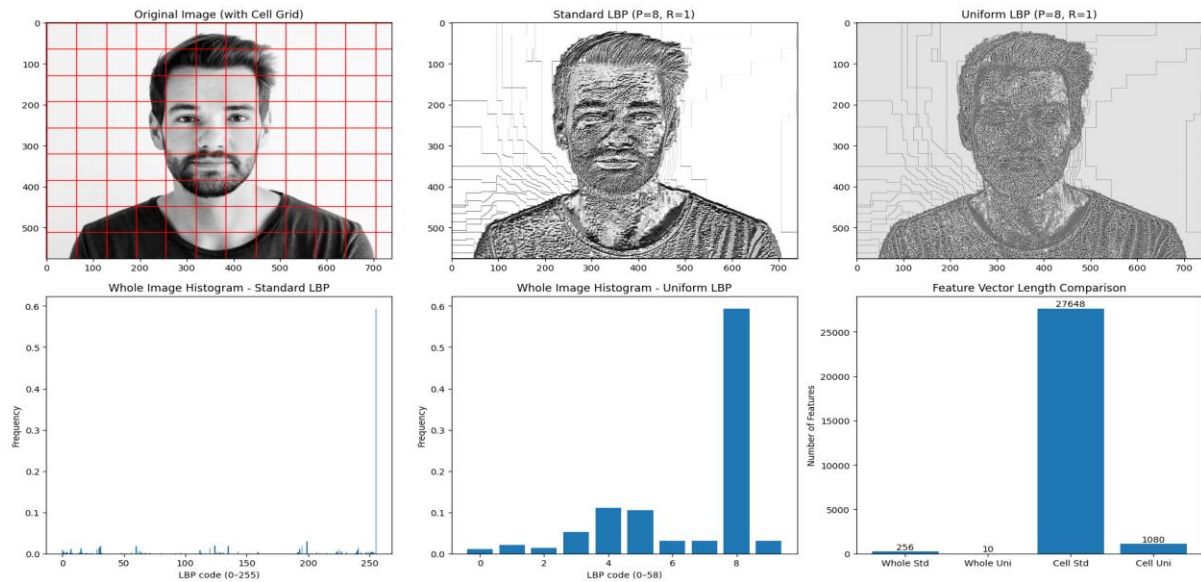
LBP Value	Count
248	1
240	1
60	1
48	1

All others (0–255)	0
--------------------	---

So the histogram (feature vector) has **256 bins**, but only four nonzero in this tiny example.

Step 5: The Whole-Image Feature Vector

Final **whole-image feature vector length** = 256 (for standard).



Whole Image LBP Feature Vector:

[0.01299268 0.02280875 0.01737519 0.05581363 0.1147241 0.10078125
0.02969219 0.02879129 0.54907798 0.06794294]

Length: 10

☒ Cell-wise LBP Feature Vector:

[0.00000000e+00 0.00000000e+00 0.00000000e+00 ... 4.34027778e-04
9.54427083e-01 0.00000000e+00]

Length: 1080

Variant of LBP

Local Ternary Pattern

Local Ternary Pattern Techniques:-

The result of LBP algorithm is grounded on L=2 means binary number operation by specifying the center pixel intensity value as a threshold. As the threshold value is center pixel, this is the main reason that LBP descriptor is more sensitive to noise specially for smooth regions of the images. To overcome this noise sensitivity problem an interesting abstraction of LBP known as is Local Ternary Pattern proposed by Tan et al [12].

In LTP, the three quantization levels were used to encode the difference between central pixel and its neighbor to a threshold. Generally the threshold value set to be manually, and its value depends in order to attain the best performance in specific problems and applications. The threshold equation for ternary operation is given below.

$$s(x) = \begin{cases} 1 & \text{if } w_p > w_c + t \\ 0 & \text{if } w_c - t \leq w_p \leq w_c + t \\ -1 & \text{if } w_p < w_c - t \end{cases} \quad (13)$$

Where w_c and w_p are defined in equation (2),(3) . t is a threshold value that controls the sensitivity of the ternary pattern against noise. To reduce the size of ternary code, the scheme is spited into two binary threshold function: upper and lower function. The upper pattern is denoted with h_i and can be computed as:

$$h_i = \begin{cases} 1 & \text{if } a_i = 1 \\ 0 & \text{otherwise} \end{cases}, (1 \leq i \leq P) \quad (14)$$

Upper / Higher LBP: Captures patterns **brighter than** the center pixel beyond threshold T. “bright” or positive contrast regions (edges, highlights). Represents “dark” or negative contrast regions (shadows, dark textures).

What these -1 shows: We use -1 in LTP to: Represent dark neighbors relative to the center pixel, preserve sign information of local contrast (bright vs dark), And reduce noise sensitivity by introducing a tolerance zone.

The binary threshold function l_i for the lower pattern is calculated:

$$l_i = \begin{cases} 1 & \text{if } a_i = -1 \\ 0 & \text{otherwise} \end{cases}, (1 \leq i \leq P) \quad (15)$$

The weighted sum function for upper pattern can be expressed as:

$$LTPh_{p,R} = \sum_{i=1}^P h_i \times 2^{i-1}, (h_i \in \{0,1\}) \quad (16)$$

The weighted sum function for lower pattern can be expressed as:

$$LTPl_{p,R} = \sum_{i=1}^P l_i \times 2^{i-1}, (l_i \in \{0,1\}) \quad (17)$$

The final LTP operator is constructed with the concatenation of the lower and upper patterns. The computational process of LTP on 3×3 image patch is represented in Figure 2.

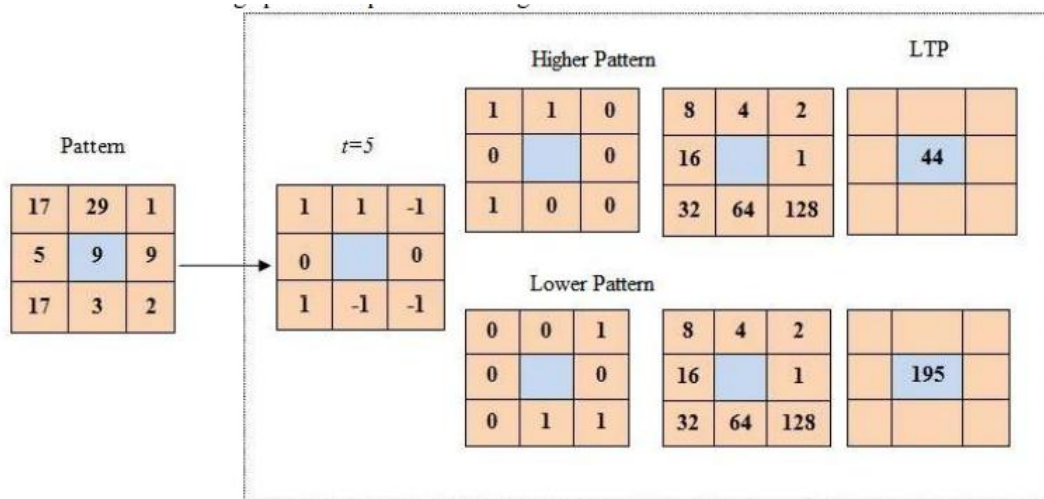


Fig 2:- Calculation of LTP operators. The obtained ternary pattern is further coded into higher and lower binary code in the LTP

Example:

108	110	115
105	107	103
100	102	106

Let the **center pixel** $I_c=107$, $I_c = 107$, $I_c=107$
and **threshold** $T=5$, $T=5$, $T=5$.

Now compute $w_p - w_c$ for each neighbor:

Neighbor	$(w_p - w_c)$	LTP Code
108	+1	0
110	+3	0
115	+8	+1
105	-2	0
103	-4	0
100	-7	-1
102	-5	-1
106	1	

0 0 +1

0 C 0

-1 -1 0

Upper (High) LBP:

Convert:

- +1 $\rightarrow$ 1
- else $\rightarrow$ 0

Start from here

108	110	115
05	107	103
100	102	106

backwise

$$S(x) = \begin{cases} 1 & w_p > w_c + t \\ 0 & w_p - t \leq w_c \leq w_p + t \\ -1 & w_p < w_c - t \end{cases}$$

$$t = 5$$

for

$$108 = w_p$$

check

$$C_1: 108 > 112 \text{ No}$$

$$C_2: 102 \leq 108 \leq 112 \text{ Yes} \rightarrow 0$$

$$C_3: 108 \leq 102 \text{ No}$$

Here

$$w_c + t \Rightarrow 107 + 5 \Rightarrow 112$$

$$w_c - t \Rightarrow 107 - 5 = 102$$

for 110

$$110 > 112 \text{ No}$$

$$102 \leq 110 \leq 112 \text{ Yes} \rightarrow 0$$

$$110 < 102 \text{ No}$$

for 115

$$115 > 112 \text{ yes} \rightarrow 1$$

$$102 \leq 115 \leq 112 \text{ No}$$

$$115 < 102 \text{ No}$$

for 103

$$103 > 112 \text{ No}$$

$$102 \leq 103 \leq 112 \rightarrow \text{yes} \rightarrow 0$$

$$103 < 102 \text{ No}$$

for 106

$$106 > 112 \text{ No}$$

$$102 \leq 106 \leq 112 \text{ yes} \rightarrow 0$$

$$106 < 102 \text{ No}$$

for 102

$$102 > 112 \text{ No}$$

$$102 \leq 102 \leq 112 \text{ yes} \rightarrow 0$$

$$102 < 102 \text{ No}$$

for 100

$$100 > 112 \text{ No}$$

$$102 \leq 100 \leq 112 \text{ No}$$

$$100 < 102 \text{ yes} \rightarrow -1$$

for 105

$$105 > 112 \text{ No}$$

$$102 \leq 105 \leq 112 \rightarrow \text{yes} \rightarrow 0$$

$$105 < 102 \text{ No}$$

$S(x) \Rightarrow$

these values
are a_i^o

0	0	1
0	0	0
-1	0	0

Binary
↓ weights

128	64	32
1		16
2	4	8

Now dividing it into

H_i^o and L_i^o
↓

if $|a_i^o| = 1 \rightarrow 1$ so
else $\rightarrow 0$

0	0	1
0	0	0
0	0	0

$[00100000] \rightarrow 32$
32 16 8 4 2 1

for L_i^o if $a_i^o = -1 \rightarrow 1$
else $\Rightarrow 0$

0	0	0
0	0	0
1	0	0

$[00000010] \Rightarrow 2$

$LTP = [H_i^o, L_i^o] \Rightarrow [32, 2]$